SYSTEM PROJECT: Student Record Management System Documentation

ER Diagram

	students	
PK	id	INT AUTO_INCREMENT
UQ	student_number	VARCHAR(20) NOT NULL
	first_name	VARCHAR(100) NOT NULL
	last_name	VARCHAR(100) NOT NULL
	course	VARCHAR(100) NOT NULL
	year_level	INT NOT NULL
	created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

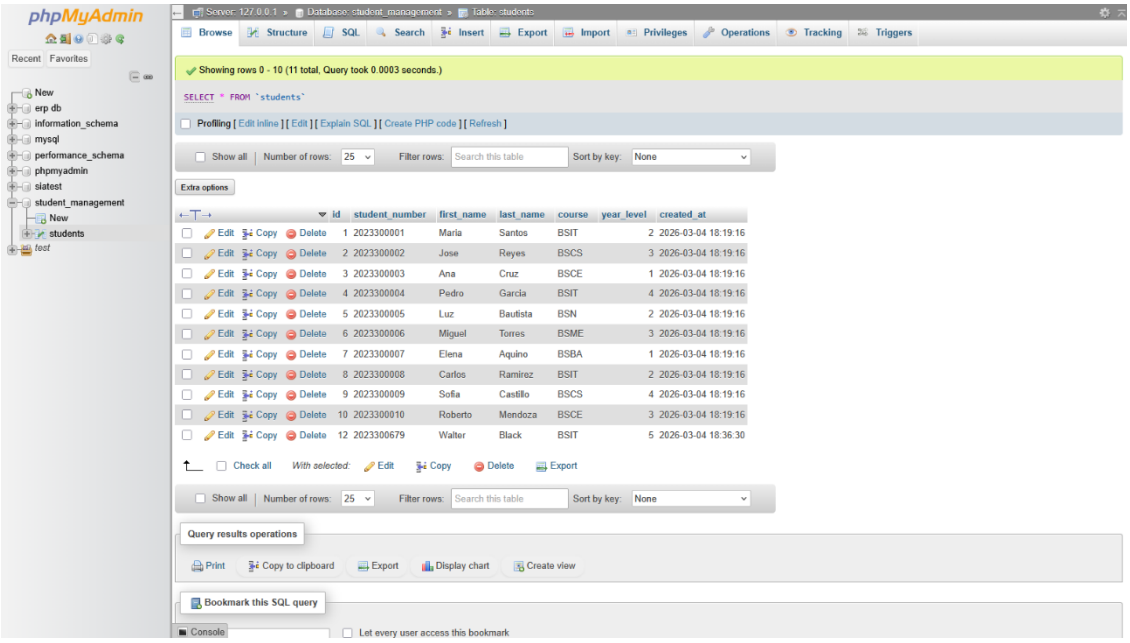
Legend:
PK = Primary Key (Unique identifier for each record)
UQ = Unique Constraint (No duplicates allowed)

API Documentation

Method	Endpoint	Description	Status Codes
POST	/api/students	Add new student	201, 400, 500
GET	/api/students	Get all students	200, 500
GET	/api/students/:id	Get single student	200, 400, 404, 500
GET	/api/students/search/:key word	Search by name/ID	200, 500
PUT	/api/students/:id	Update student	200, 400, 404, 500
DELETE	/api/students/:id	Delete student	200, 400, 404, 500

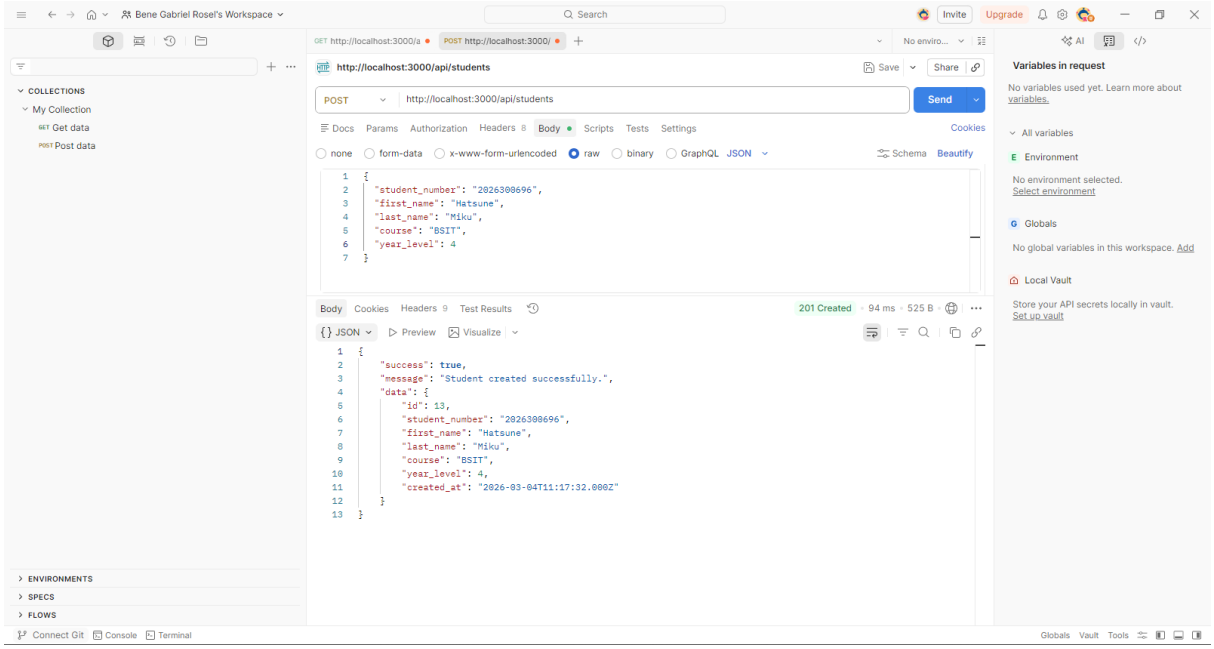
Screenshots

• Database Table

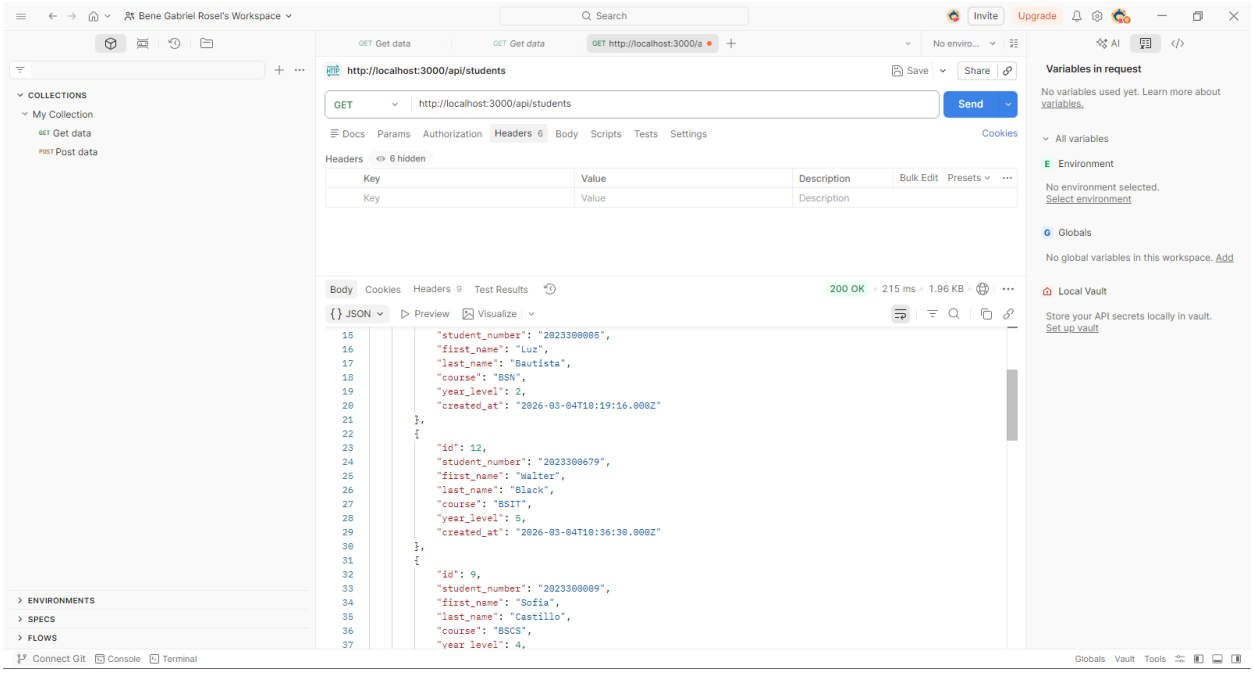


• Postman Testing

I. Insert Student (POST)

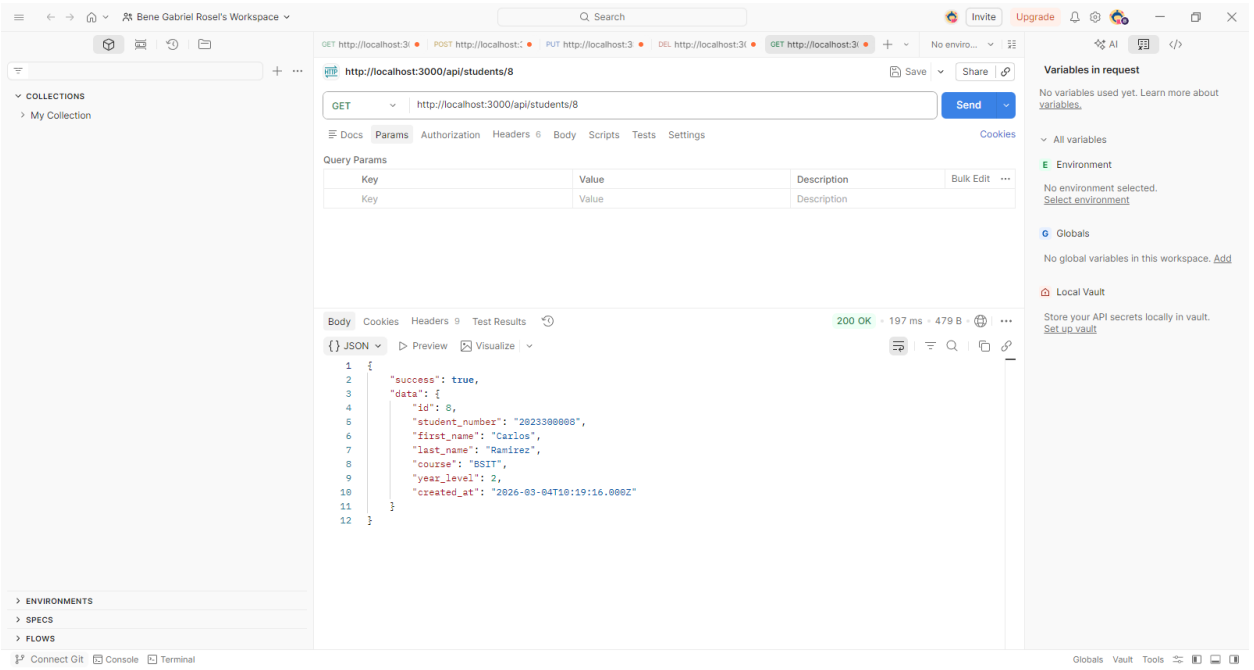


II. Select All Students (GET)

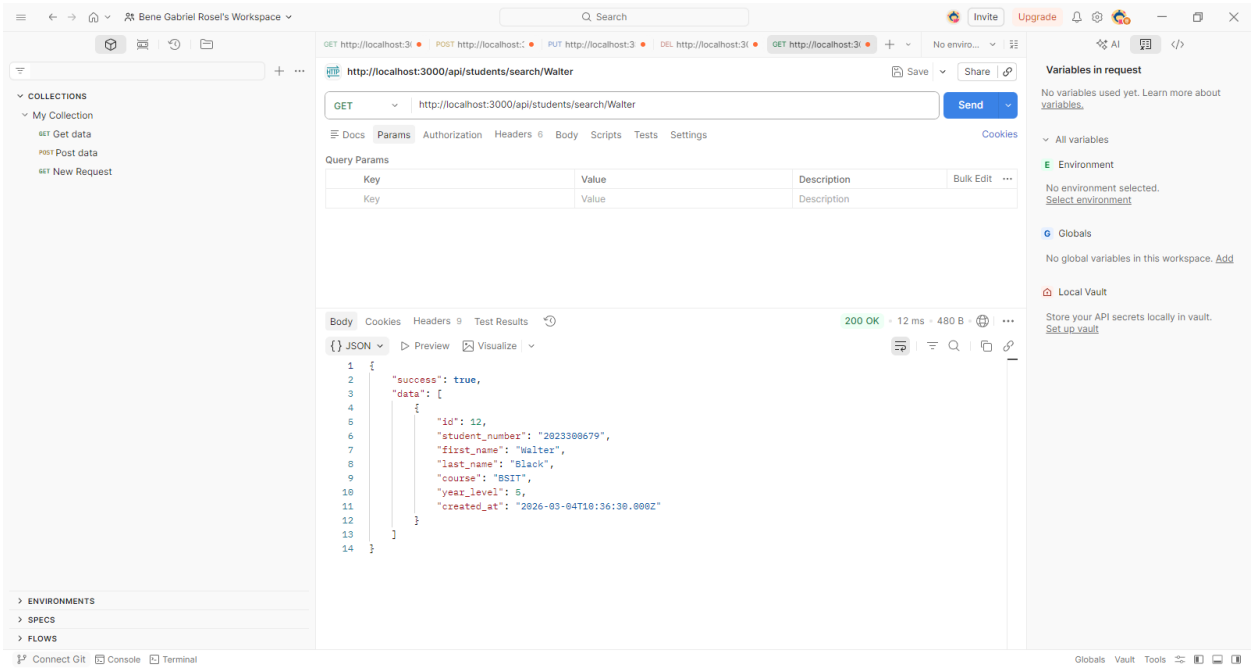


III. Select Student (GET)

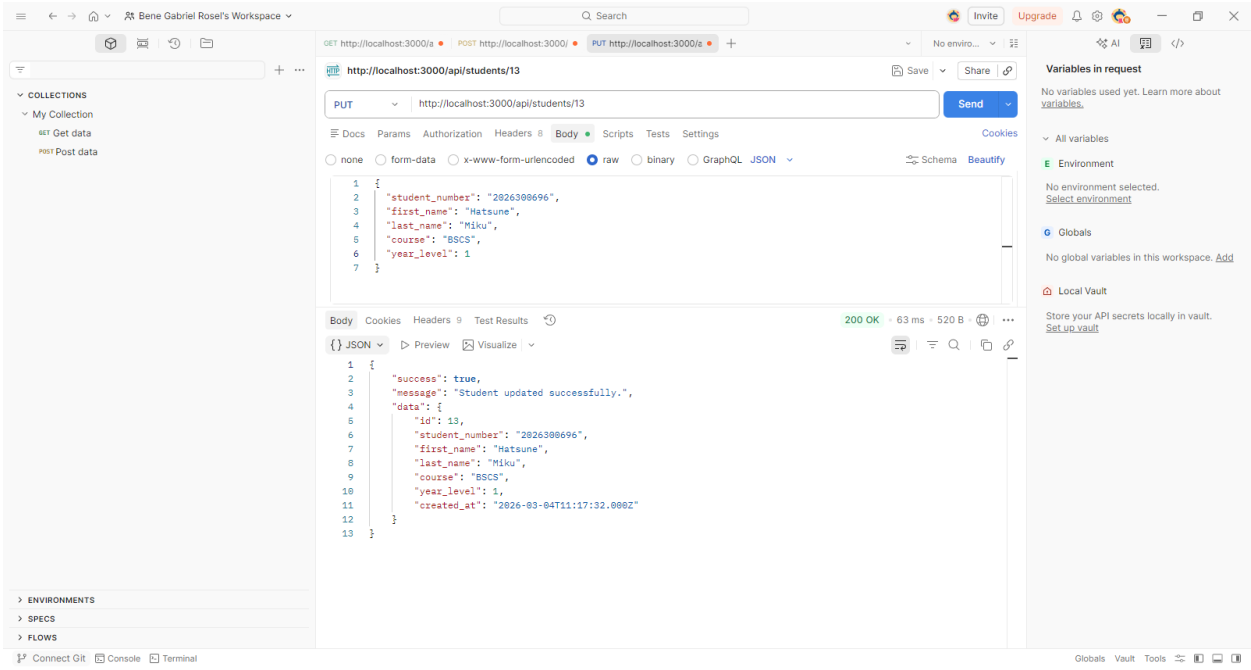
- By ID



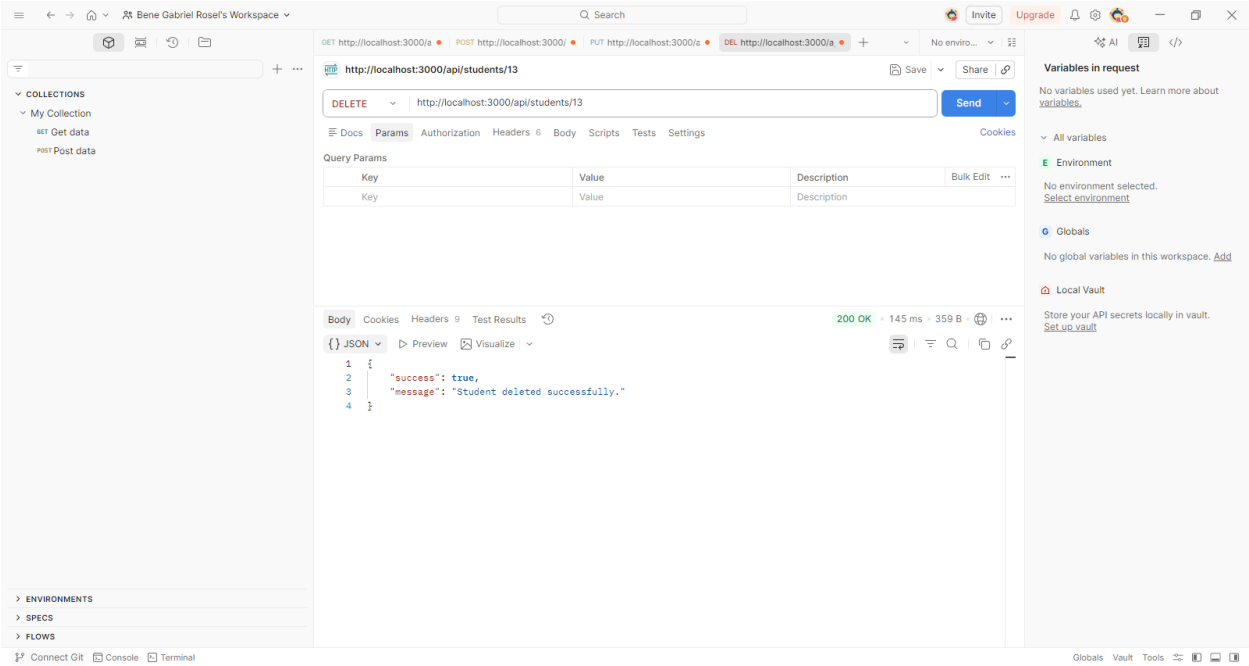
- By name/ID



IV. Update Student (PUT)



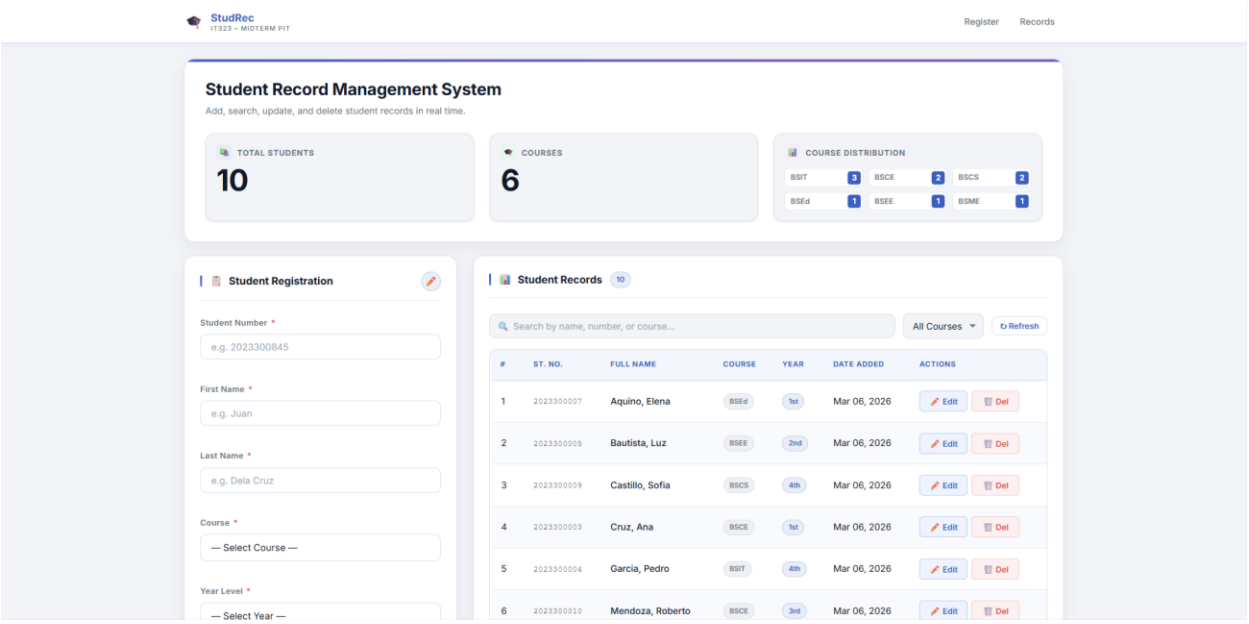
V. Delete Student (DELETE)



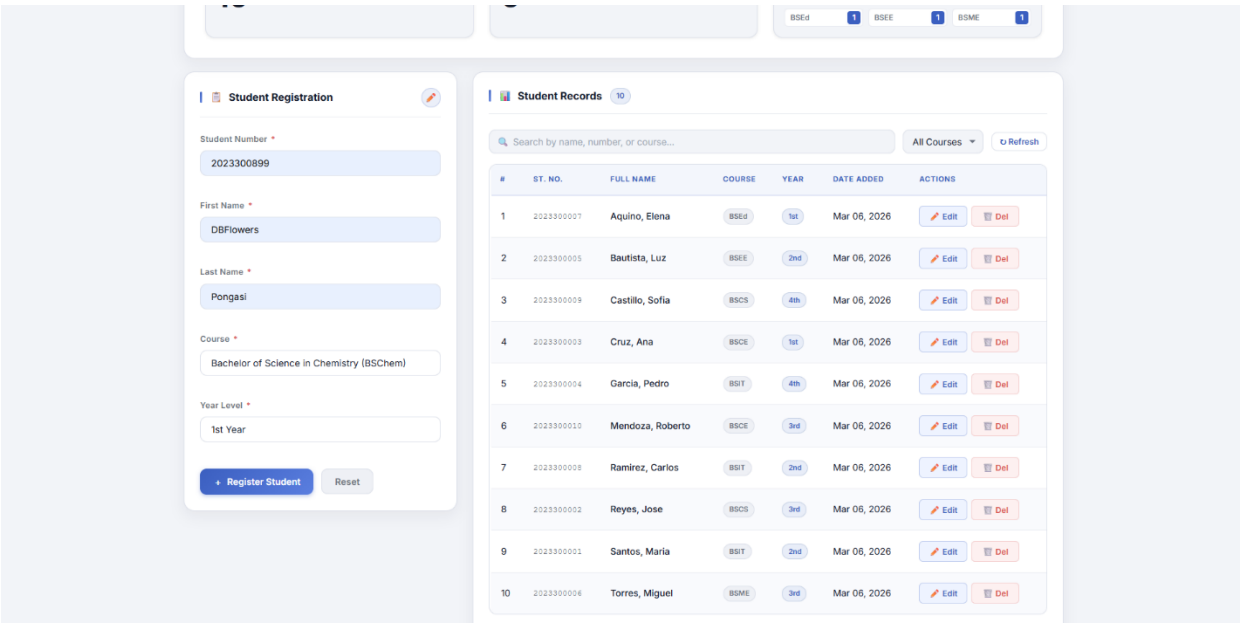
- **Front-End Design and Responsiveness**

I. Desktop

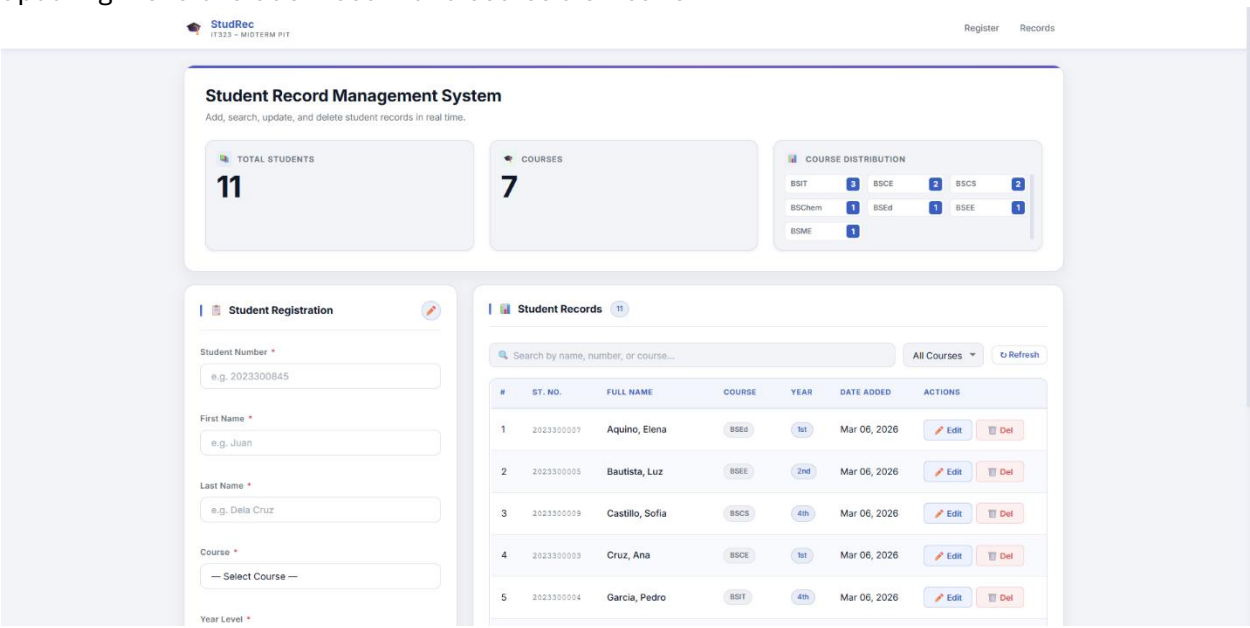
1. Main Dashboard Interface: The main dashboard showing system statistics, the student registration form, and the student records table.



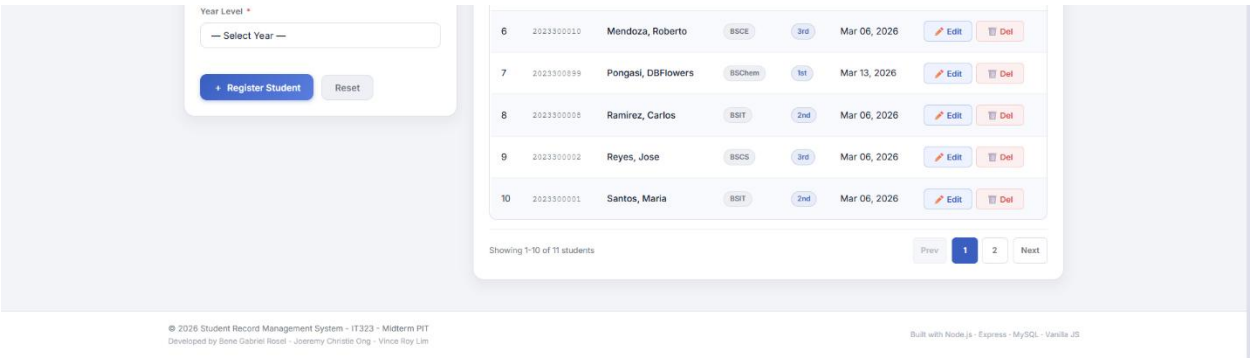
2. Student Registration: The administrator enters the student number, name, course, and year level to register a new student record.



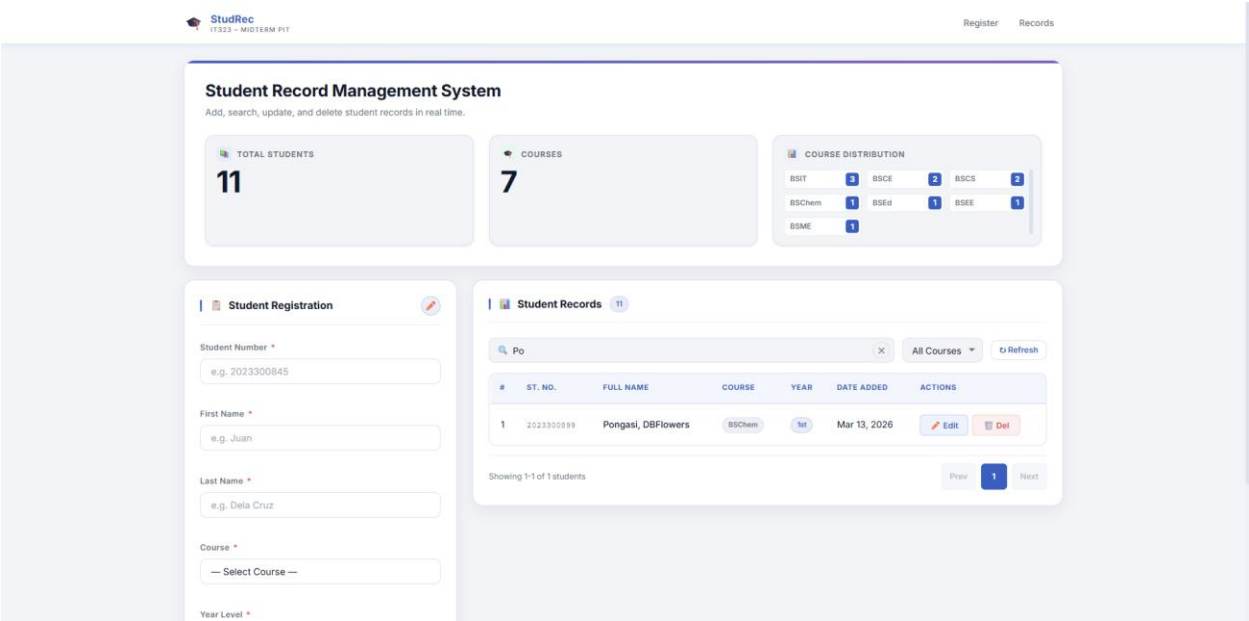
3. After Registering Student: The table automatically refreshes after a new student is added, updating the total student count and course distribution.



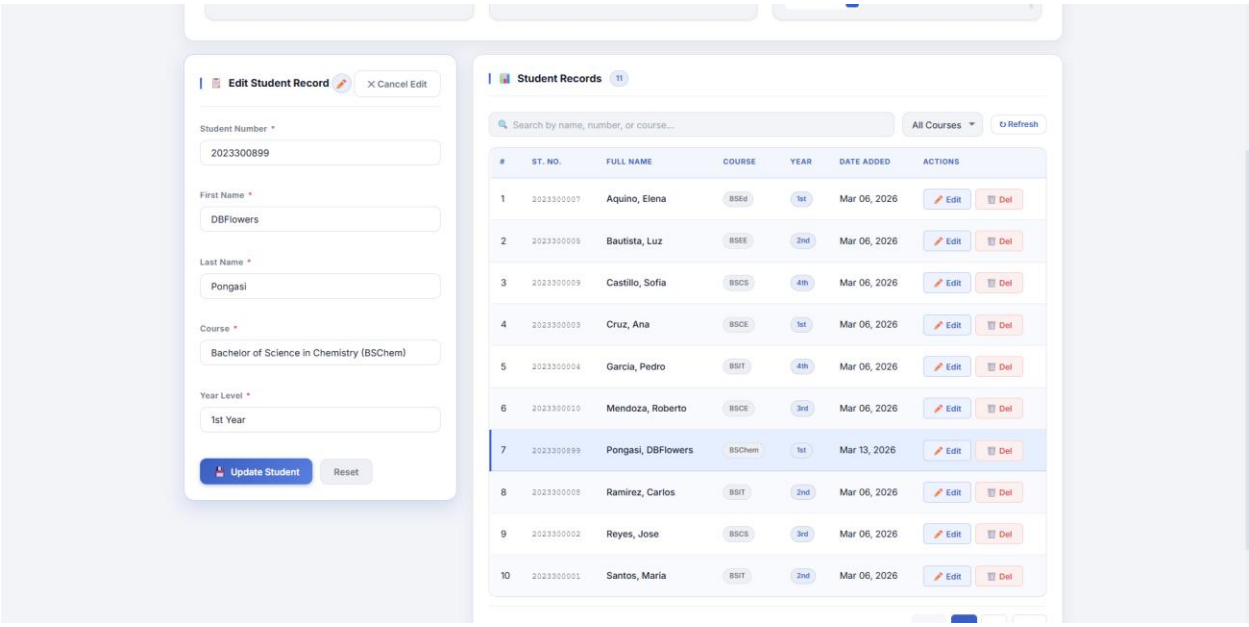
4. Student Records Table with Pagination and Footer: The table supports pagination for easier navigation when multiple student records exist.



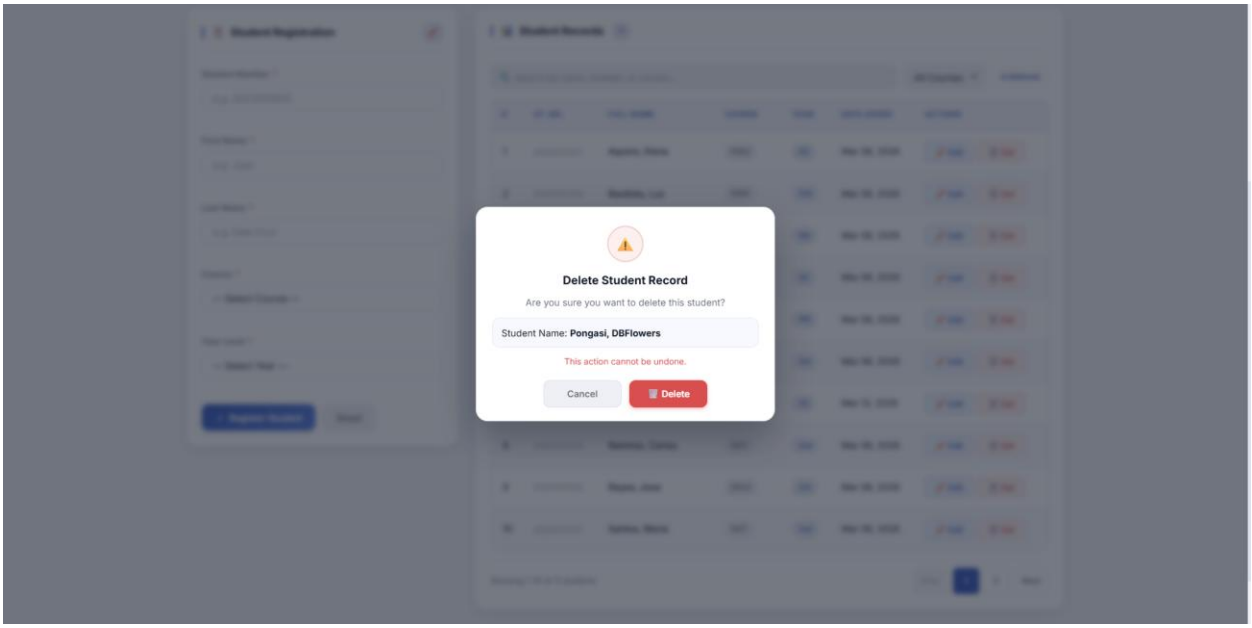
5. Student Search Functionality: The system dynamically filters student records based on name, student number, or course.



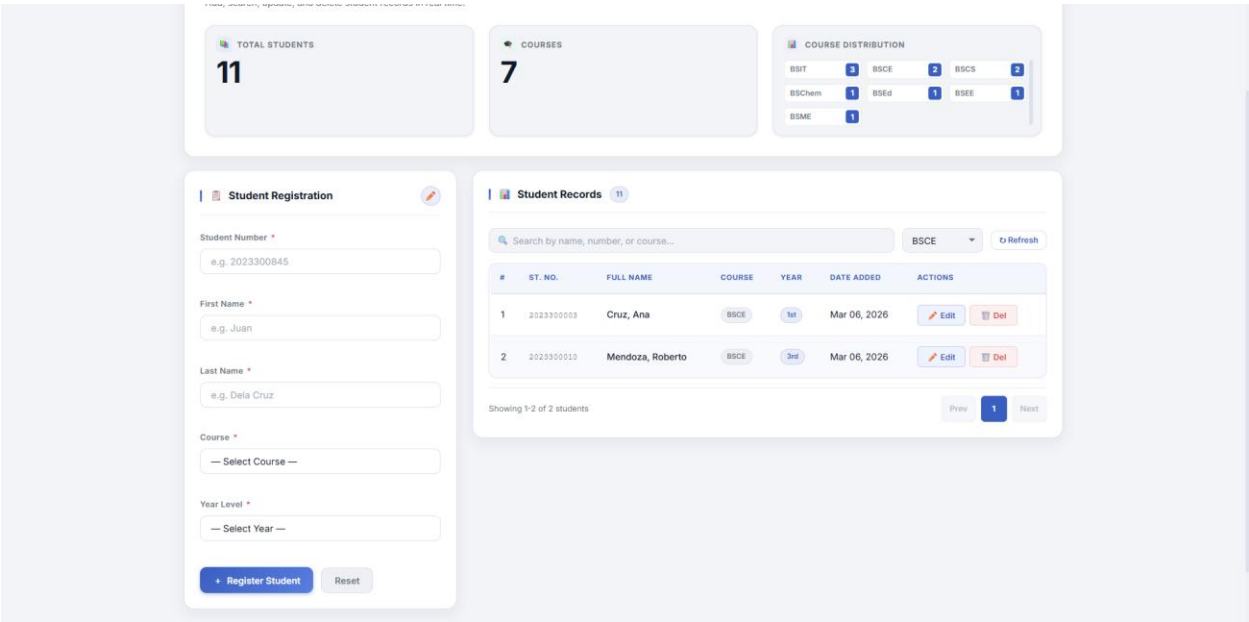
6. Edit Student Record: Clicking the Edit button populates the registration form with the selected student's data, allowing the administrator to update the record.



7. Delete Student Record Confirmation: A confirmation modal appears before deleting a student record to prevent accidental removal.

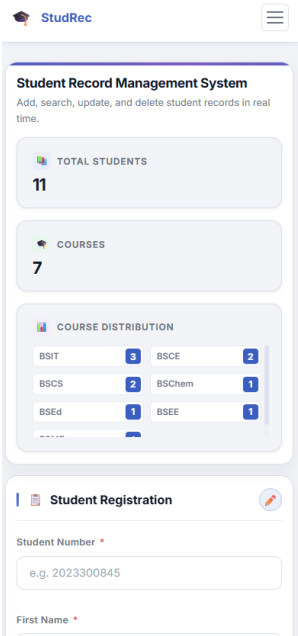


8. Filter Students by Course: Administrators can filter the table by course using the dropdown filter.

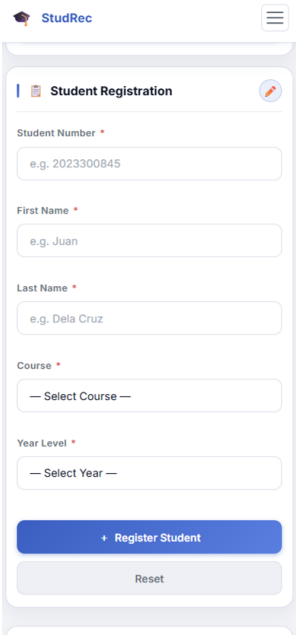


II. Mobile Responsiveness

1. Dashboard

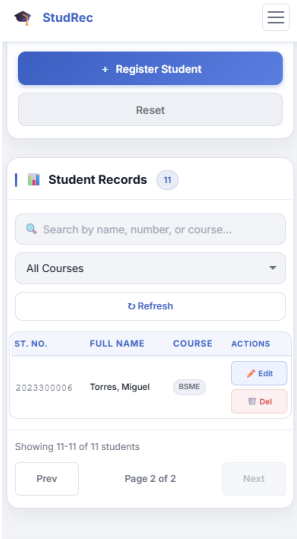


2. Student Registration

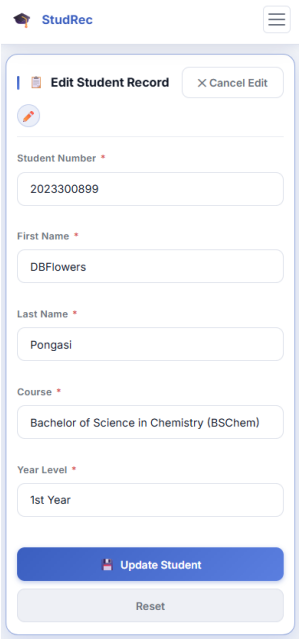


3. Student Records

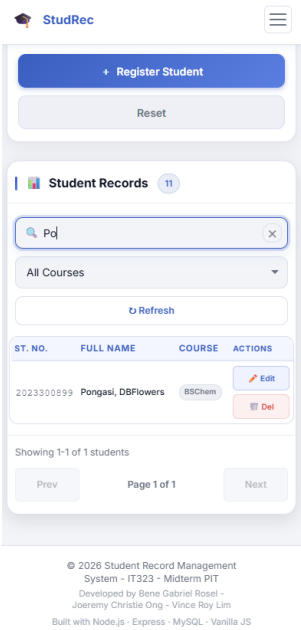
4. Pagination



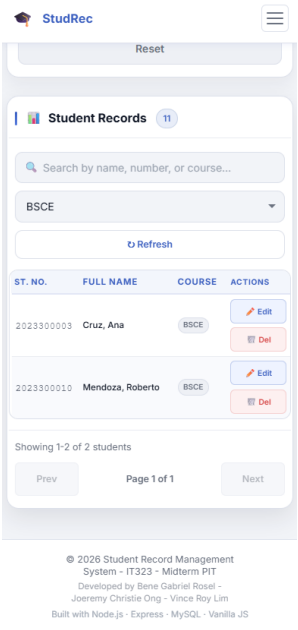
5. Edit Student Record



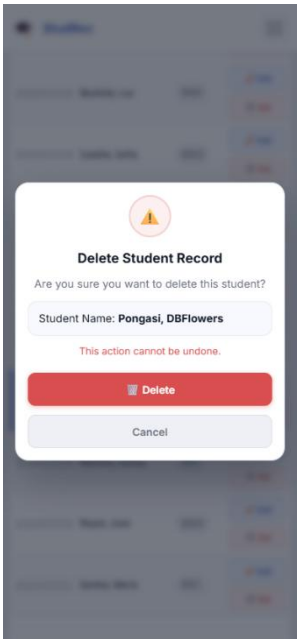
6. Search



7. Filter by Course



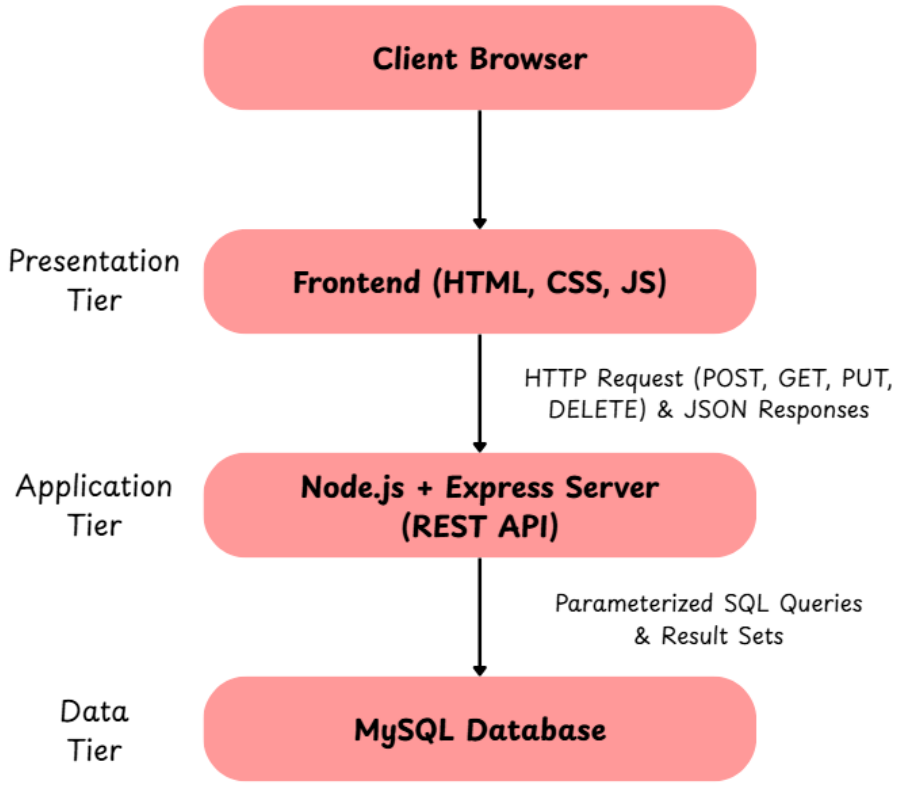
8. Delete Confirmation



Technical Report

• System Architecture

The Student Record Management System (SRMS) is a full-stack web application built following **the three-tier architecture** model:



Project Structure

- server.js - Entry point, mounts middleware + routes
- package.json - Dependencies and npm scripts
- .env.example - Environment variable template (copy to .env)
- config/db.js - MySQL connection pool (mysql2/promise + dotenv)
- routes/students.js - All /api/students route handlers
- public/
  index.html - Single-page frontend
  css/style.css - Full design system (Grid + Flex)
  js/script.js - All Vanilla JS logic
- database/schema.sql - Database schema
- database/seed.sql - Sample data
- docs/IT323-Midterm-PIT-Documentation.pdf - Project documentation

- **Database Design Rationale**

I. Schema Design

Column	Data Type	Constraints	Description
id	INT	AUTO_INCREMENT PRIMARY KEY	Unique row identifier
student_number	VARCHAR(20)	UNIQUE NOT NULL	Official student ID number
first_name	VARCHAR(100)	NOT NULL	Student's first name
last_name	VARCHAR(100)	NOT NULL	Student's last name
course	VARCHAR(100)	NOT NULL	Enrolled Academic Program
year_level	INT	NOT NULL CHECK (year_level BETWEEN 1 AND 5),	Current year level (1-4)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

- The AUTO_INCREMENT primary key ensures each record has a unique, system-generated identifier that is safe to use in API routes.
- The UNIQUE constraint on student_number enforces business rules at the database level, preventing duplicate enrollment entries even if application- level validation is bypassed.

II. SQL Operations implemented

Operation	SQL statement used
Insert student	INSERT INTO students (student_number, ...) VALUES (?, ?, ...)
Select all student (ordered ASC)	SELECT * FROM students ORDER BY last_name ASC
Select student by ID	SELECT * FROM students WHERE id = ?
Update student	UPDATE students SET student_number=?, first_name=?, last_name=?, course=?, year_level=? WHERE id=?
Delete student	DELETE FROM students WHERE id = ?
Search (using LIKE)	SELECT * FROM students WHERE first_name LIKE ? OR last_name LIKE ? OR student_number LIKE ? OR course LIKE ?

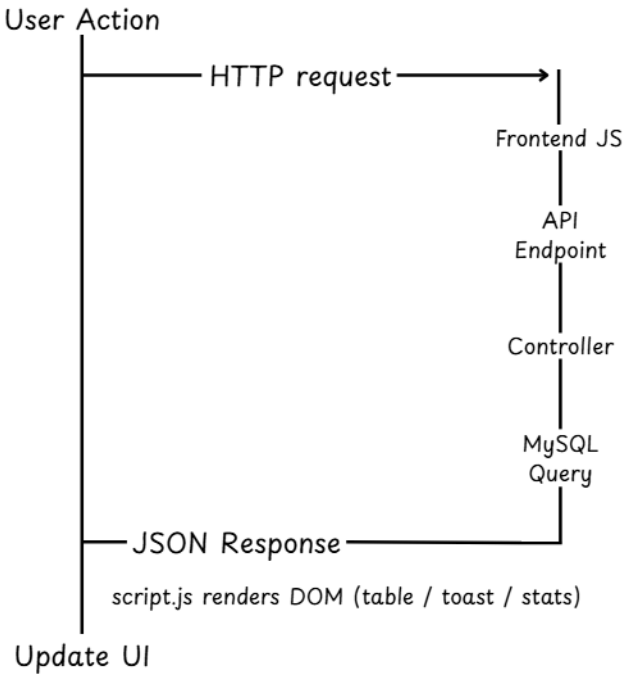
III. Normalization

1NF: All columns are atomic; no repeating groups

2NF: No partial dependencies (single-column PK)

3NF: No transitive dependencies; `course` is a free-text attribute intentionally not normalized into a separate table at this stage

- **API Flow Explanation**



- Error Handling Strategy
  400 - Missing required fields or duplicate `student_number` (caught via `ER_DUP_ENTRY` MySQL error code)
  404 - Row lookup before update/delete; returns early with message
  500 - Unexpected DB errors; caught in `try/catch`, logged server-side, generic message returned to client

- **Challenges Encountered**

- Problem: Express matched `/api/students/search/:keyword` as `/api/students/:id` (interpreting `"search"` as an ID), triggering a MySQL type mismatch error.
  Solution: The `search/:keyword` route was declared **above** the `/:id` route in `routes/students.js`. Express evaluates routes in registration order, so placing the specific literal path first ensures it matches before the generic wildcard.
- Problem: Firing an API call on every keystroke caused excessive server requests and visual flickering.
  Solution: Implemented a 400 ms debounce timer in `script.js` using `clearTimeout` / `setTimeout`. The search API is only called once the user pauses typing, reducing network calls by ~80% during normal use.
- Problem: Rendering student data directly via `innerHTML` with unsanitized data could allow stored XSS if malicious content was inserted into the database.

Solution**:** An `escHtml()` utility function was implemented to escape `&`, `<`, `>`, `"`, and `'` characters before any user-provided data is inserted into the DOM via `innerHTML`.

- Problem: At viewport widths ≤768px, the navigation links overlapped with the brand and badge elements, breaking the layout.
  Solution: A hamburger button (`#nav-toggle`) was added to the navbar. On mobile, the nav links are hidden by default and revealed as a full-width vertical dropdown when the hamburger is clicked. The button animates into a close (✕) icon when the menu is open using CSS-only bar transforms.

• Future Improvements

- User Authentication
  Implement JWT-based authentication with role-based access control (admin vs. registrar) using `jsonwebtoken` and `bcrypt` for password hashing.
- Normalized Courses Table
  Create a separate `courses` master table with a foreign key from `students.course_id`, enabling course management (add/archive courses) without modifying student records directly.
- Data Export
  Allow administrators to export the student list as a CSV or PDF report via a `GET /api/students/export` endpoint using `json2csv` or `pdfmake`.
- Automated Testing
  Add unit tests for route handlers using `Jest` + `supertest`, and integration tests against a dedicated test database to ensure regression safety during development.

Members:

Ong, Joeremy Christie

Rosel, Bene Gabriel

Lim, Vince Roy